

Master's Thesis

**AI Application Implementation and
Software-Stack Improvements
for Energy-Efficient Workloads on a CGLA**

Takuto Ando

July 30, 2026

Graduate School of Science and Technology
Nara Institute of Science and Technology

A Master's Thesis
submitted to the Graduate School of Science and Technology,
Nara Institute of Science and Technology
in partial fulfillment of the requirements for the degree of
MASTER of ENGINEERING

Takuto Ando

Thesis Committee:

Professor Yasuhiko Nakashima	(Supervisor)
Professor Yuichi Hayashi	(Committee Member)
Visiting Professor Renyuan Zhang	(Committee Member)
Assistant Professor Yirong Kan	(Committee Member)
Assistant Professor Pham Hoai Luan	(Committee Member)
Assistant Professor Le Vu Trung Duong	(Committee Member)

AI Application Implementation and Software-Stack Improvements for Energy-Efficient Workloads on a CGLA*

Takuto Ando

Abstract

Graphics processing units (GPUs) provide high AI inference throughput but consume high power, whereas fixed-function accelerators require redesign for new models and quantization formats. Motivated by this trade-off, this thesis examines the CPU-Grounded Linear Array (CGLA), a programmable accelerator architecture, using its IMAX3 FPGA prototype. We integrated a CGLA back-end into llama.cpp, whisper.cpp, and stable-diffusion.cpp. Through their shared GGML matrix-multiplication interface, quantized and FP16 operations are dispatched to format-specific CGLA dot-product kernels. For dispatched quantized operations, the PE pipeline unpacks values, applies scales, and accumulates dot products, while application control and unsupported operations remain on the CPU.

Measurements of Qwen3 model inference show that repeated data transfers and host CPU processing constrain decode. Under the 28nm ASIC projection, CGLA achieves $226\times$ better PDP (Power-Delay Product) and $51.2\times$ better EDP (Energy-Delay Product) than RTX 4090 for Qwen3-0.6B Q3_K_S at 8:1. PDP reductions are also observed in the automatic speech recognition and image generation evaluations, extending the results beyond LLM inference.

Keywords:

LLM, software stack, AI accelerator, CGLA

*Master's Thesis, Graduate School of Science and Technology,
Nara Institute of Science and Technology, July 30, 2026.

Contents

List of Figures	iv
List of Tables	iv
1 Introduction	1
1.1 Background	1
1.2 Research Objective	2
1.3 Thesis Contributions	2
1.4 Organization of the Thesis	3
2 Related Work, CGLA Architecture, and IMAX Prototype	4
2.1 AI Inference Workloads	4
2.2 Computation Platforms for AI Inference	5
2.3 From General-Purpose Devices to CGLA	5
2.4 CGLA Architecture and IMAX Prototype	8
3 GGML-Based Execution Stack and CGLA Integration	10
3.1 GGML-Based Application Stack	10
3.2 CGLA Integration Boundary	11
3.3 Execution Phases and Measurement View	12
4 LLM Execution on IMAX	14
4.1 Target Workloads and Execution Model	14
4.2 Quantized Kernel Mapping	15
4.3 System-Level Evaluation Results	17
4.3.1 Prompt Processing and Decode	19
4.3.2 Model-Size Scaling	21
4.3.3 LMM Capacity Design Space	22

5	Software Stack Improvements for CGLA	24
5.1	Need for Stack-Level Improvement	24
5.2	Q-Snap	24
5.2.1	Chunk-Size Derivation	25
5.2.2	Quantitative Results	26
6	Speech Recognition and Image Generation on the IMAX Pro-	
	tototype	28
6.1	Extension to GGML-Based Applications	28
6.2	Whisper ASR	28
6.2.1	FP16 Kernel Design and Implementation Techniques . .	29
6.2.2	Energy Efficiency and Platform Comparison	31
6.3	Stable Diffusion	34
6.3.1	Experimental Setup and Quantitative Results	34
6.4	Cross-Workload Discussion	36
6.5	Evaluation Scope and Limitations	36
7	Conclusion	37
	Acknowledgements	38
	References	39
A	Publications	44
B	Awards	45

List of Figures

2.1	Manycore/GPGPU and CGLA structures.	6
2.2	Overview of the IMAX prototype and CGLA lane structure.	7
3.1	Three-layer software stack view.	11
4.1	Task partitioning for LLM inference.	15
4.2	Q6_K and Q3_K dot-product dataflows.	17
4.3	End-to-end latency for Qwen3 configurations.	19
4.4	Phase breakdown for prompt processing and decode on IMAX.	20
4.5	Latency and PDP for Qwen3-1.7B Q8_0.	22
4.6	PDP and EDP for Qwen3-0.6B Q3_K_S.	22
4.7	LMM-size PDP sweep for Qwen3 workloads.	23
5.1	Comparison of Q-Snap, the default heuristic, and fixed-size chunking.	25
5.2	Throughput improvement of Q-Snap.	26
5.3	Chunk-size sensitivity of Q-Snap.	27
6.1	FP16 dot-product kernel dataflow on the IMAX prototype.	30
6.2	Whisper ASR processing flow.	31
6.3	PDP and burst-length trade-off for Whisper ASR.	32
6.4	LMM capacity trade-off for Whisper ASR.	34
6.5	IMAX and RTX 4090 comparison for SD-Turbo.	35

List of Tables

4.1	Specifications of the Qwen3 evaluation platforms and projected IMAX ASIC.	18
6.1	Cumulative Whisper kernel coverage by LMM size.	33

1 Introduction

1.1 Background

The increasing size of AI models has made inference energy a system-level concern. LLM inference, speech recognition, and image generation repeatedly execute matrix operations, move model parameters, and invoke host CPU control. These operations make total execution energy relevant in addition to device peak power.

Graphics processing units (GPUs) are widely used for LLM inference and other AI workloads due to their high throughput and mature software stacks, yet incur substantial power consumption. Application-specific integrated circuits (ASICs) specialized for fixed functions reduce power but cannot readily accommodate changes in model structure or quantization format. To address this limitation, our prior work proposed IMAX, a CPU-Grounded Linear Array (CGLA) accelerator that arranges processing elements (PEs) and local memory modules (LMMs) in a one-dimensional structure [1, 2]. IMAX3 is the FPGA prototype used to evaluate CGLA in this thesis. The implementation and evaluation use existing open-source software (OSS) applications under local, low-batch conditions and include dominant kernels, transfer granularity, and host CPU control.

The three evaluation targets are llama.cpp for LLM inference [3], whisper.cpp for speech recognition [4], and stable-diffusion.cpp for image generation [5]. They share a common GGML computation backend. The CGLA backend offloads dominant GGML kernels while preserving existing front-end flows.

1.2 Research Objective

This thesis implements CGLA execution paths for LLM inference, automatic speech recognition (ASR) with Whisper, and image generation with Stable Diffusion. The implementation includes backend dispatch, transfer policy, fallback handling, and phase-wise profiling for CGLA execution.

The evaluation addresses three questions.

1. Which application, offload-layer, and CGLA components are required to execute the three GGML-based applications without replacing their front ends?
2. In the Qwen3 paths, which measured phases constrain prompt processing and decode, and how does Q-Snap change prompt-processing throughput?
3. How does the shared GGML interface support quantized and FP16 operations for Whisper and Stable Diffusion, and what correctness, latency, and PDP results follow under the documented conditions?

Chapter 3 defines the common measurement interval and phase decomposition used to answer these questions.

1.3 Thesis Contributions

The main contributions are listed below.

- It implements CGLA execution paths within llama.cpp, whisper.cpp, and stable-diffusion.cpp while retaining their model loading, tensor management, command-line interfaces, and application control.
- It implements quantized dot-product offload and phase-wise profiling in llama.cpp, where input transfer and host processing account for the dominant decode costs in the Qwen3 configurations.
- It implements Q-Snap as a stack-level policy, increasing Qwen3-0.6B Q3_K_S prompt-processing throughput by a factor of 1.62 under the tested configuration.

- It implements FP16 offload for Whisper. On 21 LibriSpeech test-clean utterances with Whisper-tiny.en, the mean normalized CPU-IMAX transcript deltas are 0.00% for FP16 and 0.13% for Q8_0.
- It reuses the backend for Stable Diffusion Q3_K, Q8_0, and FP16 dot products, with the tested FP16 shapes matching the CPU reference without guard-region corruption.

1.4 Organization of the Thesis

Chapter 2 gives the architectural and research background. Chapter 3 defines the software-stack viewpoint and the measurement boundary used in the evaluation chapters. Chapter 4 analyzes LLM execution on IMAX. Chapter 5 evaluates Q-Snap as a software-stack improvement within the same llama.cpp path. Chapter 6 evaluates Whisper ASR and Stable Diffusion as additional application paths. Chapter 7 concludes the thesis.

2 Related Work, CGLA Architecture, and IMAX Prototype

Workload and platform requirements motivate the CGLA architecture and IMAX prototype described below.

2.1 AI Inference Workloads

Modern LLM inference is dominated by matrix multiplication, weight movement, and runtime control. The transformer architecture made multi-head attention and large feed-forward blocks central to language modeling [6]. Later model scaling increased both compute and memory demand [7]. Prompt processing tends to contain larger matrix operations, whereas decode repeatedly executes smaller operations and is more sensitive to transfer overhead and fixed costs. This phase difference matters for CGLA because transfer granularity and host control can dominate performance even when the arithmetic kernel is accelerated.

The llama.cpp stack combines quantization, backend selection, tensor management, and application-level inference [3]. Post-training quantization methods such as GPTQ [8], LLM.int8() [9], and AWQ [10] reduce the memory footprint of LLM execution [11]. GGML supports multiple quantization formats, allowing models at different precisions to be loaded and executed from a single stack.

Whisper ASR and Stable Diffusion provide application cases with data types and runtime structures that differ from LLM inference. Whisper [12] is a transformer encoder-decoder model for speech recognition, and whisper.cpp provides its C/C++ inference path [4]. Stable Diffusion follows the Latent Diffusion Model

architecture [13], and `stable-diffusion.cpp` provides the C/C++ inference path used here [5]. Both models rely on the same GGML backend as `llama.cpp`, so the CGLA integration can reuse the open-source tensor and backend interfaces. These workloads differ from LLM inference but share common integration challenges in host CPU control, local-memory capacity, direct memory access (DMA) behavior, and kernel coverage.

2.2 Computation Platforms for AI Inference

CPUs provide general programmability, but their energy efficiency is limited for dense tensor operations. GPUs provide high throughput and mature ecosystems but consume high power. Wide single-instruction, multiple-data (SIMD) units, large register files, and high-bandwidth memory hierarchies optimized for throughput all contribute to power dissipation. High-bandwidth memory (HBM)-equipped GPUs increase memory throughput, but in the low-batch inference cases studied here, launch, transfer, and host-control costs can still remain in end-to-end time. AI inference is dominated by matrix multiplication and weight movement, and it tolerates reduced arithmetic precision with little accuracy loss [11]. This property motivates specialized hardware. However, evolving model structures and quantization formats make the key design trade-off one between specialization and generality.

2.3 From General-Purpose Devices to CGLA

General-purpose processors, GPUs, domain-specific architectures, and coarse-grained reconfigurable architectures (CGRAs) expose different costs in computation, data movement, and programmability.

General-purpose processors allocate area and power to out-of-order (OoO) and speculative execution. Dense, branch-free tensor loops underutilize these mechanisms. GPUs omit speculation and instead sustain many SIMD units under single-instruction, multiple-thread (SMT) execution. However, hiding warp stalls requires high-bandwidth memory, coalescing, and large register files and caches. Hardware-managed data movement simplifies programming but incurs

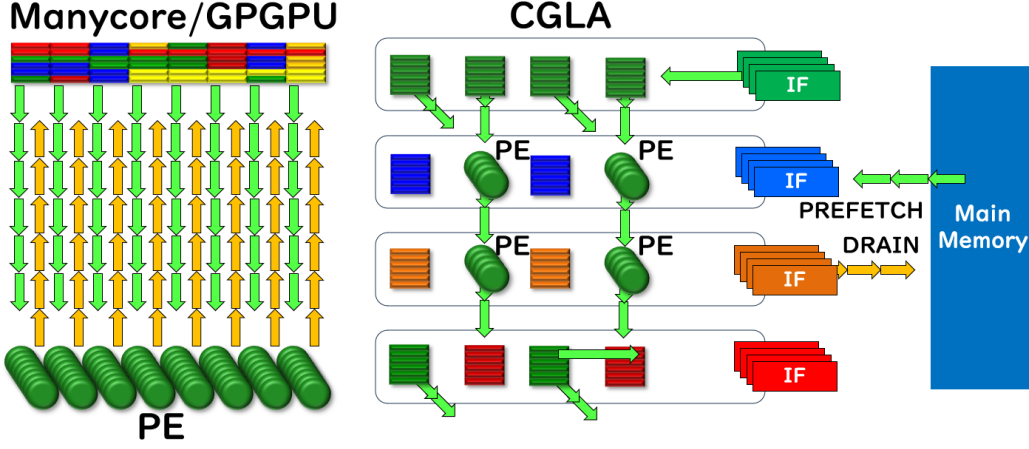


Figure 2.1. Comparison of the manycore/GPGPU and CGLA structures. The figure contrasts shared-memory delivery to many PEs with unidirectional dataflow through a linear PE/LMM arrangement.

area and power overhead on embedded targets. Fig. 2.1 contrasts the manycore and general-purpose graphics processing unit (GPGPU) structures with CGLA. In a manycore or GPGPU, many PEs receive data through a shared memory hierarchy. In contrast, CGLA arranges PEs and LMMs in one dimension and explicitly controls data movement through PREFETCH and DRAIN operations to main memory and unidirectional transfers between adjacent units.

Domain-specific architectures (DSAs) can achieve high power efficiency through low-precision fixed-point multiply-accumulate, deep compute pipelines, and data reuse that reduce main-memory traffic [14, 15]. These designs exploit the small accuracy loss from reduced-precision inference [11] and the high energy cost of main-memory access [16]. Fixed-function DSAs restrict their supported operations to a narrow target and cannot accommodate unsupported operations without hardware redesign. Redesigning an ASIC for each workload becomes impractical as fabrication costs rise. This trade-off motivates a programmable accelerator that retains software-defined mapping while reducing device power relative to a GPU.

CGRAs spatially connect coarse-grained compute blocks through topology-dependent routing networks, which can include neighbor-oriented connections [2, 17]. Systolic arrays, including the array used in the TPU [18], also use spatial data movement for regular computation but are not equivalent to general CGRAs.

Systolic arrays such as TPUs target regular dense matrix execution, while application control, quantization format, and execution shape still affect the software boundary around such kernels. The EMAX family places an LMM and address-generation logic inside each unit for stencil and vector operations [19, 20]. This organization overlaps burst transfer with computation and sustains arithmetic throughput. Two-dimensional CGRAs, however, demand extensive routing that reduces area efficiency and complicates physical implementation. Broadcasting identical data to multiple LMMs for irregular access wastes memory [20]. Fig. 2.2 illustrates the IMAX prototype system structure and CGLA PE/LMM arrangement within each lane.

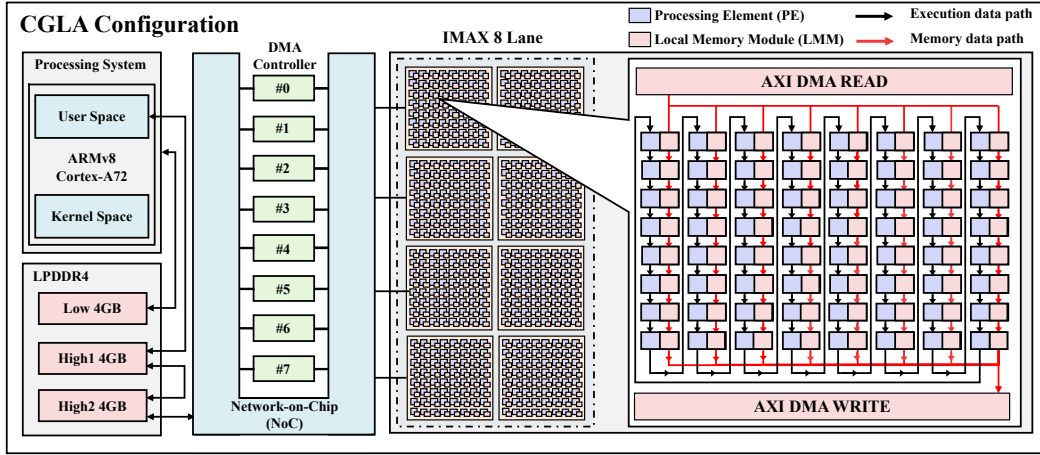


Figure 2.2. Overview of the IMAX prototype and CGLA lane structure. The figure shows the host, DMA path, multiple lanes, and the alternating PE/LMM arrangement within each lane.

IMAX instantiates the CGLA architecture by arranging PEs and LMMs alternately in a one-dimensional ring with unidirectional data transfer between adjacent units [1]. The one-dimensional layout eliminates placement and routing overhead and limits wiring load. Non-overlapping memory regions eliminate cache-line eviction and the large-scale multithreading otherwise required to hide it. Unused LMMs can be clock-gated for further power reduction. CGLA retains software-defined mapping across evolving model structures and quantization formats while targeting lower device power than GPUs. The evaluation focuses on local-memory fit, explicit DMA, and host fallback as controllable parts of the

CGLA application path. The experiments use IMAX3, a field-programmable gate array (FPGA) prototype of CGLA.

2.4 CGLA Architecture and IMAX Prototype

The IMAX prototype implements the one-dimensional CGLA PE/LMM ring introduced in Sect. 2.3 [1, 2]. Host-supplied data is stored in lane-local LMMs and processed as a pipeline flowing through adjacent PEs.

CGLA employs complex-instruction-set-computer (CISC)-style processing elements, in contrast to the reduced-instruction-set-computer (RISC)-style units of typical two-dimensional CGRAs. A CISC-based PE encodes more work per hardware operation, reducing operation sequence length and dependency depth. Kernels map to a sequence of PE-and-LMM pairs without two-dimensional placement, routing, or iterative search during compilation. The same operation sequence executes on a CPU before hardware mapping, which simplifies algorithm verification. Each kernel execution follows a fixed phase sequence. CONF configures the PE array and LMMs, and LOAD transfers input data via DMA. EXEC runs the pipeline, and DRAIN returns the results. Ideally, DRAIN of one iteration overlaps with EXEC of the current iteration and LOAD prefetch of the next, sustaining continuous pipeline occupation [1]. The throughput obtained from this overlap depends on chunk size, LMM capacity, and DMA granularity.

The IMAX implementation supports cascading chips through an Advanced eXtensible Interface (AXI) slave interface without dedicated external memory. The lineage progresses from the Linear Array Pipeline Processor (LAPP) through the Energy-aware Multimode Accelerator eXtension (EMAX) series to IMAX [1]. The generations advance LMM capacity, DMA throughput, area efficiency, and multi-chip support. The IMAX3 system comprises an AMD Versal VPK180 FPGA with on-chip ARM Cortex-A72 [21], 8 lanes, 64 PEs per lane, and 512 KiB LMM per lane. ASIC projections scale IMAX-active FPGA measurements from 145 MHz to 840 MHz using 28 nm static-timing and synthesis estimates obtained with Synopsys Design Compiler [22].

CGLA exposes data movement directly to software. This control can reduce memory-hierarchy overhead but renders local-memory capacity, DMA granular-

ity, and host-side control first-order design constraints. The same arithmetic kernel can produce different end-to-end behavior depending on chunking and transfer policy. Uetani [23] implemented LLM inference on CGLA and mapped the dominant dot-product kernels to the accelerator. Eto [24] evaluated LLaMA-style models on IMAX and characterized kernel coverage across quantization formats. Together, these studies establish functional LLM execution and kernel coverage. The present implementation adds Whisper ASR, Stable Diffusion, and Q-Snap to the GGML backend and measures their transfer, configuration, and host processing costs.

3 GGML-Based Execution Stack and CGLA Integration

The three application paths share the software structure and measurement procedure defined below. Their implementations and results appear in Chapters 4 and 6.

3.1 GGML-Based Application Stack

The implementation reuses existing OSS application stacks and modifies only the components required for CGLA integration. Model loading, tensor management, command-line interface (CLI) behavior, and core application logic remain in the original implementations. Compute-intensive kernels are selectively offloaded to the CGLA backend. The measurements therefore include integration costs outside the arithmetic kernels. End-to-end performance depends on quantization format, chunking strategy, DMA control, host-side runtime behavior, and fallback handling, not solely on arithmetic kernel efficiency.

The target is a GGML-based C/C++ stack shared by the three applications. llama.cpp and whisper.cpp use this stack. The stable-diffusion.cpp path uses the same tensor backend. These applications retain their original front-end flow. The backend selects host, CGLA, or mixed execution from the operation shape, data type, and transfer size. The measurements include runtime costs from the original front ends.

3.2 CGLA Integration Boundary

The execution stack has three layers. At the application layer, the three target applications retain their original flow. Within the offload layer, backend dispatch, chunk formation, transfer scheduling, fallback handling, and profiling are orchestrated. The CGLA hardware layer executes kernels and manages the local memory modules. Fig. 3.1 illustrates the responsibilities of each layer and the connection from GGML-based applications to the CGLA backend. The offload layer selects the execution target from operation shape and data type, then manages the associated transfers.

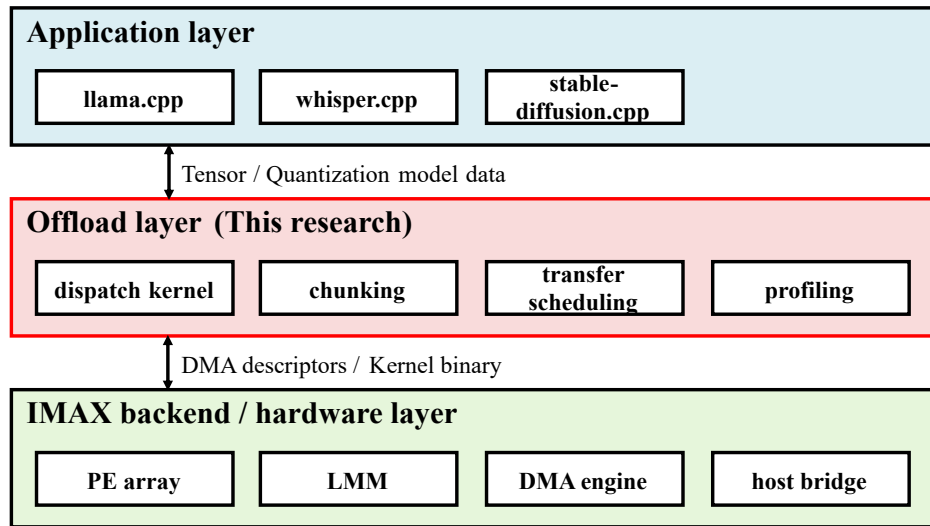


Figure 3.1. Three-layer software stack connecting llama.cpp, whisper.cpp, and stable-diffusion.cpp to the CGLA backend. Together, the layers preserve the application flow, control dispatch and transfer, and execute the selected kernels on the IMAX prototype.

An application-level quantization choice alters the data layout exposed to the offload layer. Offload-layer decisions therefore determine the frequency of transfer and control costs incurred by hardware execution. High standalone kernel throughput may still yield poor end-to-end performance.

The offload layer contains CGLA paths for LLM kernels, an FP16 path for Whisper ASR, Stable Diffusion backend insertion, Q-Snap chunk selection, and profiling hooks. Subsequent chapters evaluate these artifacts within complete

application execution paths. The stack selectively offloads kernels conforming to the linear dataflow model and executes the remainder on the host. Once a target kernel is selected, the offload layer partitions the data into chunks fitting the CGLA local-memory modules and issues corresponding transfers. Chunk boundaries govern burst size, DMA transaction count, quantization alignment, and setup overhead. Chunking is therefore treated as a runtime policy. CPU fallback handles unsupported operations, but its synchronization and control transitions can degrade end-to-end performance even when the main kernels are offloaded. Host overhead and transfer cost are coupled. Reducing one can expose the other as the dominant bottleneck.

3.3 Execution Phases and Measurement View

Measurements use two granularities. The application interval covers execution after model loading until the application produces its output. The accelerator-phase decomposition covers host configuration, DMA transfer, and PE-array computation within an offloaded kernel. CONF denotes the time the host CPU spends writing mapping commands and execution-control information to IMAX via programmed I/O (PIO). CONF is separate from LOAD of input data. Repeated configuration for short chunks remains in end-to-end time independently of arithmetic execution. REGV measures the PIO time required to initialize internal PE registers for the selected kernel. Its cost scales with the number of PEs used and the configuration state size. RANGE configures the LMM address regions assigned to input, output, and workspace data. LOAD transfers input data from host main memory to the LMMs via a contiguous DMA buffer. When input arrays are non-contiguous, the host first consolidates them into a contiguous region to minimize DMA transaction count. EXEC measures the interval during which the configured PE array reads data from the LMMs and executes the target kernel as a pipelined sequence of SIMD instructions. DRAIN transfers the result from the LMMs to the DMA buffer in host main memory.

The LMMs employ double buffering, enabling EXEC on one buffer to overlap with LOAD or DRAIN on the other buffer. Phase times thus represent residual costs under overlapped transfer and computation, not purely sequential decom-

position. A large LOAD fraction indicates limitations in input volume, DMA burst granularity, or host-to-accelerator bandwidth. A large EXEC fraction signals that the PE pipeline dominates execution time. A large DRAIN fraction reflects output layout, transaction count, or write-back path constraints. Large CONF, REGV, or RANGE fractions indicate that configuration is repeated frequently for short chunks. This decomposition distinguishes optimizations needed in arithmetic kernels from those required in chunking or host CPU control logic.

Phase-wise measurements separate compute time from transfer and host overhead. The implementation records LOAD, DRAIN, EXEC, CONF, REGV, and RANGE together with host time outside offloaded kernels. Each platform is timed from the completion of model loading until application output is produced after CPU-side postprocessing. The measured interval includes accelerator-kernel execution, accelerator configuration and transfer, host fallback, and residual host execution. Model loading is excluded because storage and page-cache state can otherwise affect the measured interval. The results represent complete platforms, including their platform-specific CPU and memory configurations.

The hardware evaluation uses end-to-end latency, phase-wise execution breakdown, power-delay product (PDP), energy-delay product (EDP), kernel coverage, and LMM-capacity sensitivity. For a platform with a single documented power consumption, PDP and EDP are defined from power consumption and end-to-end latency as

$$\text{PDP} = \text{Power consumption} \times \text{Latency}. \quad (3.1)$$

$$\text{EDP} = \text{Power consumption} \times \text{Latency}^2. \quad (3.2)$$

For the IMAX ASIC projection, the PDP is calculated as

$$\text{PDP}_{\text{IMAX}} = P_{\text{IMAX}}T_{\text{IMAX}} + P_{\text{ARM}}T_{\text{ARM}}, \quad (3.3)$$

where P_{IMAX} and P_{ARM} denote power consumption, and T_{IMAX} and T_{ARM} denote IMAX-active and residual host CPU latency, respectively. The IMAX EDP is the resulting PDP multiplied by the projected end-to-end latency. Each platform comparison keeps the model and quantization format fixed, so application quality is checked for functional equivalence before hardware metrics are interpreted [25]. The Whisper paths use word error rate (WER) and character error rate (CER), and the Stable Diffusion kernels use CPU-reference checks.

4 LLM Execution on IMAX

The llama.cpp implementation maps quantized dot-product kernels to IMAX and retains application control on the host. Its measured transfer and host costs motivate the software-stack changes in Chapter 5.

4.1 Target Workloads and Execution Model

LLM inference runs on the IMAX3 prototype through llama.cpp [3]. The CGLA backend offloads the dominant compute kernels while retaining the original host execution structure.

Measurements cover both prompt processing and decode because they use different matrix shapes and transfer frequencies. Chunking, DMA, host control, and quantization format together constrain end-to-end performance. The Qwen3 family (0.6B, 1.7B, and 8B) supplies multiple model scales within a single llama.cpp stack [26]. The Qwen3 experiments measure end-to-end latency, energy-oriented metrics, and LMM-size sensitivity. The Llama3-8B Q3_K_S experiment measures kernel coverage and host fallback behavior in the same implementation. Fig. 4.1 illustrates the inference architecture and the host-CGLA task boundary.

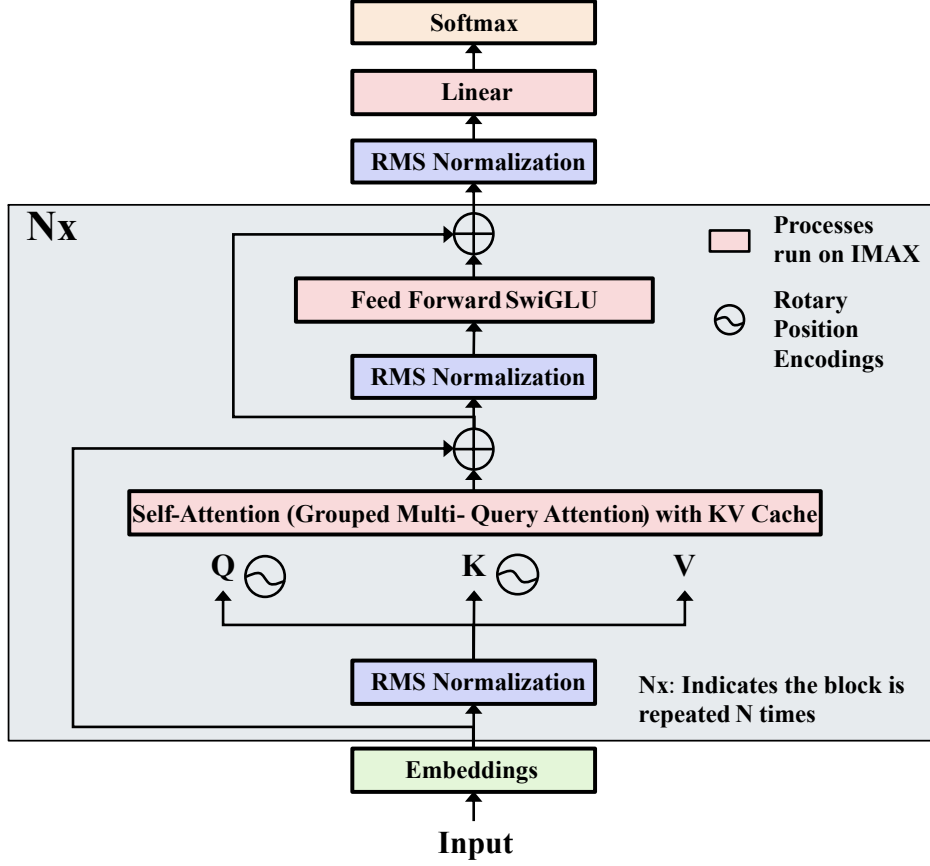


Figure 4.1. Task partitioning between llama.cpp and the CGLA backend. The host retains model management, execution control, and non-offloaded operations. The IMAX prototype executes quantized dot-product kernels invoked from GGML matrix multiplication.

4.2 Quantized Kernel Mapping

Quantized dot-product kernels in the GGML execution path are the primary targets. Q8_0, Q6_K, and Q3_K kernels are mapped onto the one-dimensional CGLA dataflow. Host-side control, tensor management, and key-value (KV) cache logic remain on CPU, while quantized dot products from matrix multiplication are offloaded.

Q8_0. Q8_0 represents each 32-element weight block with signed 8-bit values and a block scale. The kernel reads quantized weights and 8-bit inputs from the LMMs and uses SML8 to generate two 8-bit products in parallel. The partial

sums flow through the PE column as 24-bit integers, are accumulated with AD24, and are converted to FP32 after applying the block scale. Four dataflows operate in parallel, and two executions process a 32-element segment. The kernel uses 46 PEs and provides the common integer multiply-accumulate and scale-application structure reused by the lower-bit kernels.

Q6_K. Q6_K stores 256 weights as separate 2-bit high and 4-bit low components with 8-bit scales. The CVT86 instruction reconstructs these components and their scales into an intermediate representation in one cycle. The SML16 instruction performs dot products with 8-bit inputs across four parallel dataflows repeated sixteen times for 256 elements. The kernel uses all 64 PEs, increasing both PE utilization and configuration cost.

Q3_K. Q3_K stores 256 weights as 1-bit high and 2-bit low components combined with 6-bit scales. The CVT53 instruction merges the weight components into a 3-bit representation and converts the scale for the following SIMD pipeline. The kernel then uses SML16 and FP32 conversion in the same backend structure as Q6_K. Four parallel dataflows are repeated sixteen times to process 256 elements. The Q3_K mapping uses 51 PEs and performs unpacking and accumulation continuously within one PE column.

The storage costs for Q8_0, Q3_K, and Q6_K, including block scales, are 8.50, 3.44, and 6.56 bits per weight, respectively, versus 16.00 for FP16. Q3_K and Q6_K reduce storage relative to Q8_0 but demand format-specific unpacking at the front end (CVT53/CVT86). Quantization reduces LMM demand and transfer volume at the cost of additional unpacking stages in the PE column.

Fig. 4.2 compares the one-column dataflows for Q6_K and Q3_K mapped to CGLA. Both kernels decompress mixed-precision packed weights at the front end, then share a common multiply-accumulate and FP32 pipeline. Format-specific instructions adapt the unpacking and compute stages.

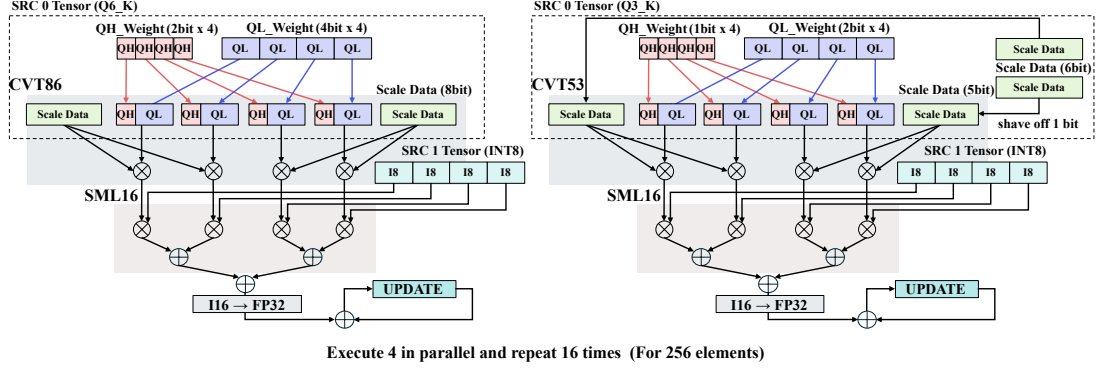


Figure 4.2. One-column dataflows of the Q6_K and Q3_K dot-product kernels mapped onto CGLA. Format-specific unpacking and scale handling are performed at the front end, followed by a shared SIMD integer multiply-accumulate and FP32 conversion pipeline.

In the Q3_K_S path, Q6_K operations accounting for approximately 10% of total computation remained on the host fallback path. Adding IMAX-side Q6_K reduced the latency for generating 32 tokens with Llama3-8B from 34.2s to 30.4s. The 3.8s difference is the measured latency change associated with adding this kernel coverage in the tested configuration.

4.3 System-Level Evaluation Results

The system-level evaluation measures execution phases after quantized-kernel of-fload and varies model scale and LMM capacity. The evaluation uses Qwen3-0.6B, Qwen3-1.7B, and Qwen3-8B in Q8_0 and Q3_K_S on the IMAX3 prototype. The hardware is a VPK180-based IMAX3 with 8 lanes, 64 PEs per lane, 512 KiB LMM per lane, a 145 MHz FPGA clock, and two host threads. For ASIC projection, IMAX-active FPGA measurements are scaled from 145 MHz to 840 MHz using the 28 nm Design Compiler estimates [22]. Host CPU execution remains unscaled. PDP and EDP use the resulting execution times, synthesis-based IMAX estimates, and nominal platform power estimates. Each horizontal-axis label pairs the number of prompt tokens with the number of generated tokens. Each comparison uses the same host-thread setting and IMAX prototype configuration within the reported Qwen3 condition. Table 4.1 summarizes the Qwen3 evalu-

Table 4.1. Specifications and power models of the Qwen3 evaluation platforms and projected IMAX ASIC. The ASIC projection uses the 28 nm Design Compiler estimates.

Device	Host CPU	Compute units ^a	Process (node)	Freq. (MHz)	Memory	Power model ^c (W)
IMAX3 (VPK180) [21]	Cortex-A72	64	7	145	8 GB+4 GB ^b	–
IMAX (28 nm, 1 lane) [1]	–	64	28	840	–	0.647
RTX 4090 [27]	Xeon W5-2455X [28]	16384	4N	2520	24 GB	650
GTX 1080 Ti [29]	Xeon W5-2455X	3584	16	1582	11 GB GDDR5X	250
Jetson AGX Orin [30]	Cortex-A78AE	1792	8	930	32 GB	15

^aFor IMAX3 and IMAX, compute units denote PEs per lane. For RTX 4090, GTX 1080 Ti, and Jetson AGX Orin, they denote CUDA cores. ^b8 GB DDR4 OS buffer and 4 GB DDR4 DMA buffer. ^cValues used in the recalculated PDP/EDP model.

ation platforms and the projected IMAX ASIC specifications from the IMAX report and manufacturer documentation. The archived GTX 1080 Ti latency series in Fig. 4.3 and Fig. 4.5 uses the Xeon W5-2455X host listed in Table 4.1 and the model, quantization, and token configurations shown in the figures. Its 250 W nominal board-power value is reported for traceability, but the current recalculated PDP and EDP plots exclude GTX 1080 Ti because their power model includes only projected IMAX, RTX 4090, and Jetson AGX Orin. In the Qwen3 configurations, data transfer and host-side overhead are the primary constraints on decode execution. Increasing clock frequency or lane count alone therefore does not make end-to-end latency scale proportionally when LOAD, CONF, DRAIN, and residual host time remain. Fig. 4.3 presents end-to-end latency for Qwen3-0.6B Q3_K_S and Qwen3-8B Q8_0. The 8B case exhibits larger latency gaps against GPU platforms and a greater weight-transfer volume.

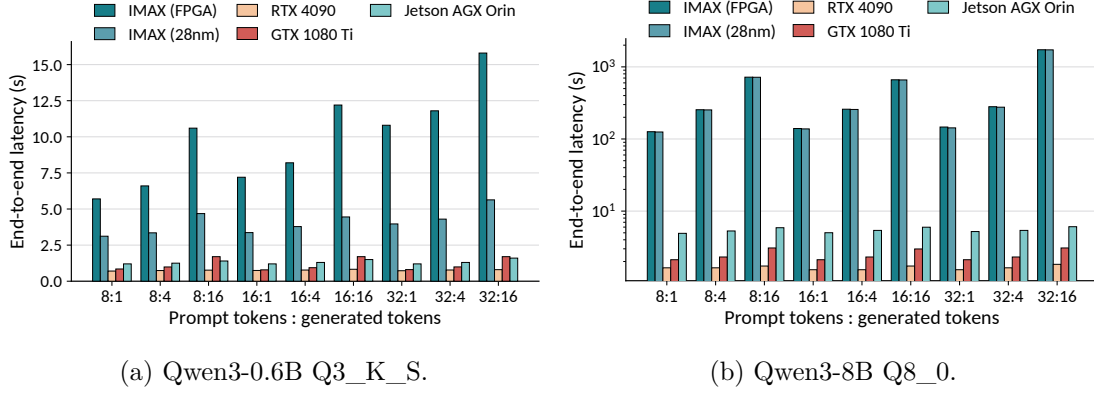
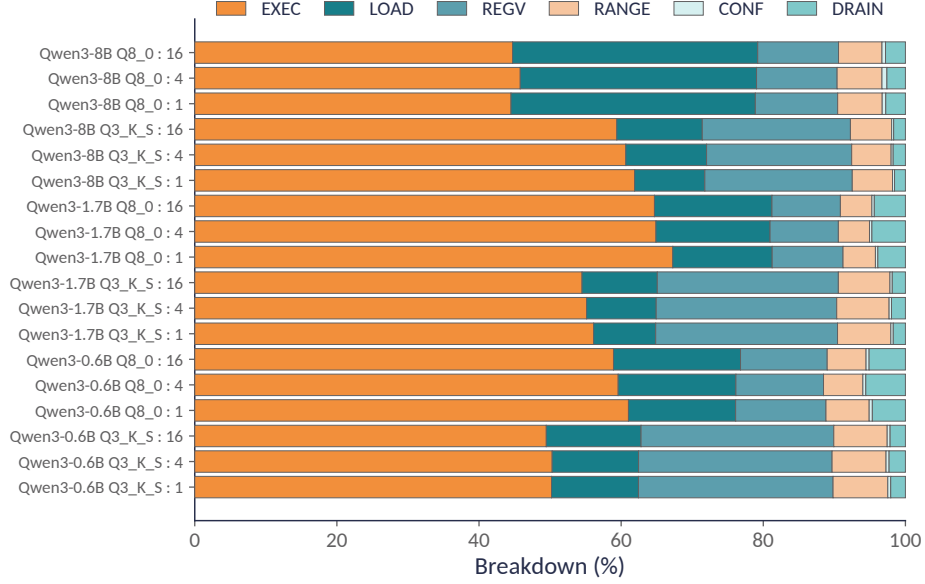


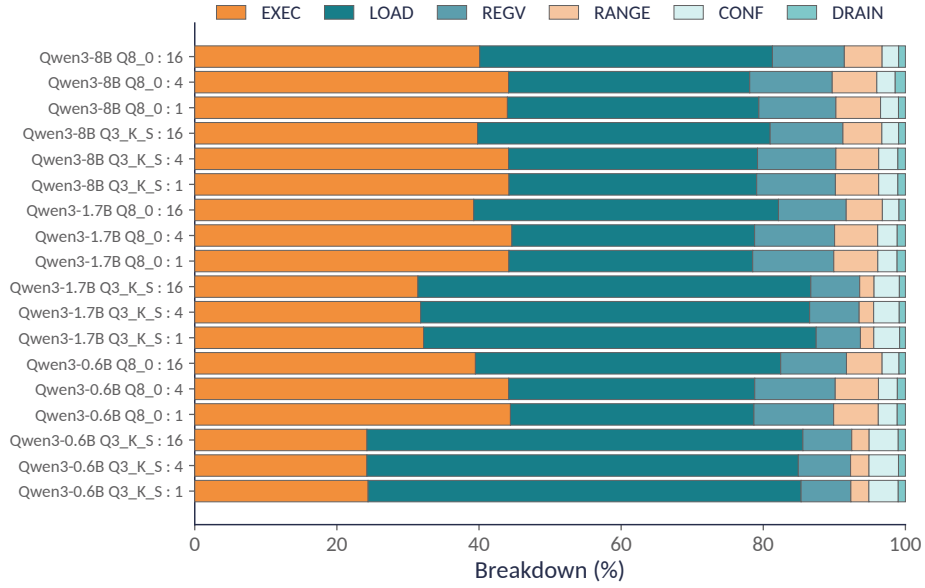
Figure 4.3. End-to-end latency for the small Qwen3-0.6B Q3_K_S and large Qwen3-8B Q8_0 configurations. Each horizontal-axis label pairs the number of prompt tokens with the number of generated tokens.

4.3.1 Prompt Processing and Decode

Fig. 4.4 presents the execution-time breakdown for prompt processing and decode. Prompt processing amortizes transfer costs over larger matrix operations, while decode repeatedly invokes smaller operations and exposes fixed host overhead. In the decode measurements, LOAD accounts for the dominant accelerator phase because each generated token requires parameter re-transfer. The phase distributions therefore require different optimization targets. Reducing EXEC can shorten prompt processing, but it does not remove the transfer and host costs exposed by repeated decode steps. Reducing end-to-end decode time requires policies that reduce transfer volume or the number of transfer and configuration events through chunk formation and transfer scheduling.



(a) Prompt-processing phase.



(b) Decode phase.

Figure 4.4. Execution-time breakdown by phase. Prompt processing has a larger EXEC fraction, whereas decode has a larger LOAD fraction because input transfer repeats for each generated token.

The prompt–decode asymmetry means that a single static execution policy does not address both phases. For variable generated lengths, the application uses chunk formation, padding, or residual fallback. End-to-end latency depends on chunk occupancy and transfer count. Chapter 5 addresses this gap through software-stack modifications.

4.3.2 Model-Size Scaling

The evaluation spans three model sizes of 0.6B, 1.7B, and 8B parameters. Fig. 4.5 shows end-to-end latency and recalculated PDP for the 1.7B model under Q8_0. Fig. 4.6 reports PDP and EDP for the representative Qwen3-0.6B Q3_K_S case.

For Qwen3-1.7B Q8_0, RTX 4090 has the shortest latency, while the projected IMAX has the lowest PDP in every configuration shown in Fig. 4.5. The PDP advantage therefore comes from lower estimated power rather than shorter execution time. For Qwen3-0.6B Q3_K_S, the projected IMAX has the lowest PDP and EDP across the nine prompt–generation configurations in Fig. 4.6. Both metrics increase as the generated sequence becomes longer because transfer and residual host time also increase. The stronger latency weighting in EDP reduces the advantage as these non-kernel costs grow. For the 8:1 configuration in Fig. 4.6, the projected IMAX achieves $226\times$ better PDP and $51.2\times$ better EDP than RTX 4090 under the stated power model.

Across the Qwen3 model-size sweep, the parameter count increases from 0.6B to 8B, and weight-transfer volume grows accordingly. Prompt processing shows higher EXEC fractions with larger matrix dimensions. Decode exhibits increasing per-token transfer cost because each token requires more weight data through DMA. Consequently, the LOAD fraction grows across the model sizes, reinforcing the decode-phase transfer dominance shown in Fig. 4.4.

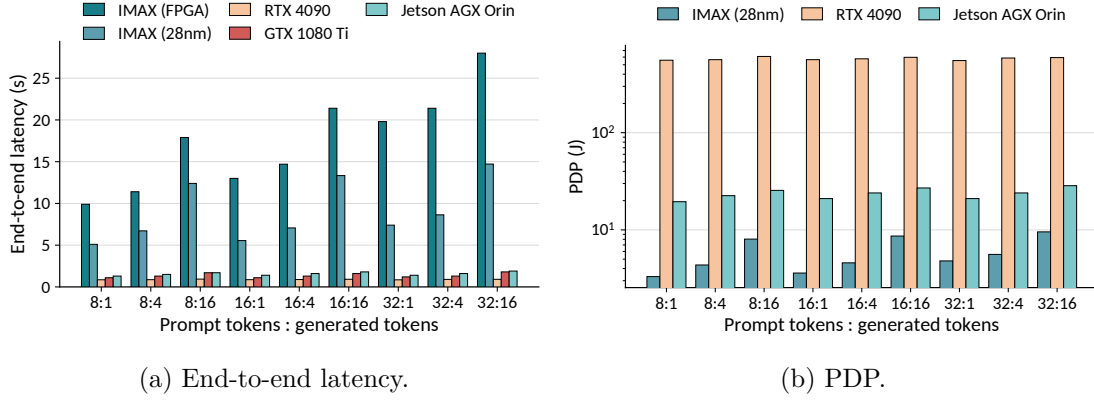


Figure 4.5. End-to-end latency and recalculated PDP for Qwen3-1.7B Q8_0.

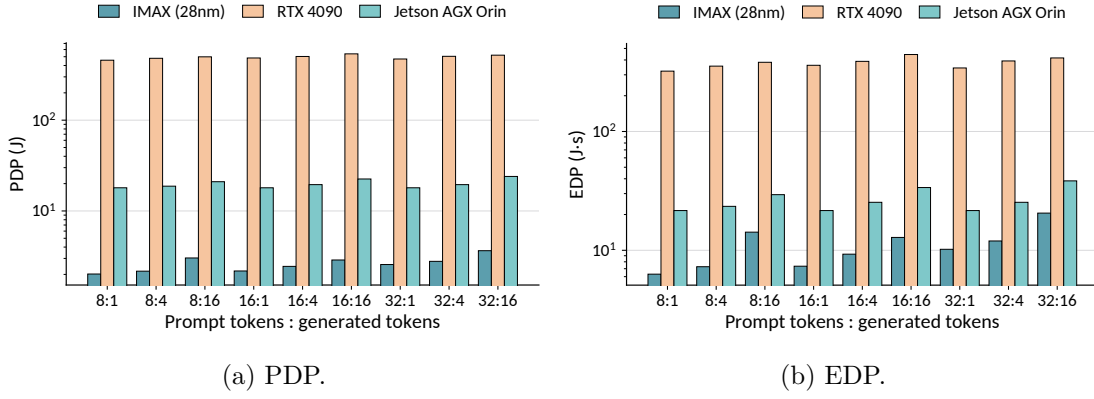
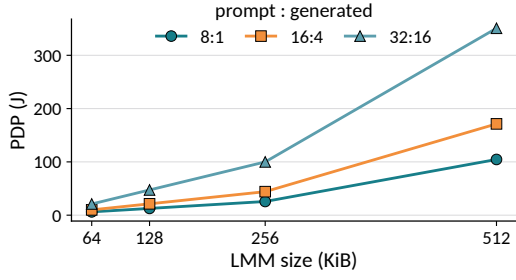


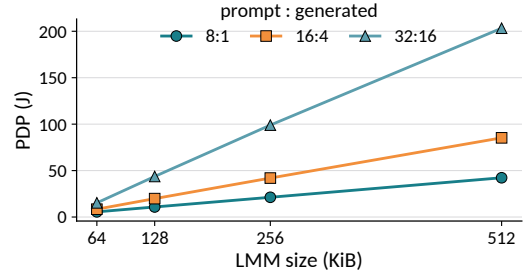
Figure 4.6. Recalculated PDP and EDP for Qwen3-0.6B Q3_K_S.

4.3.3 LMM Capacity Design Space

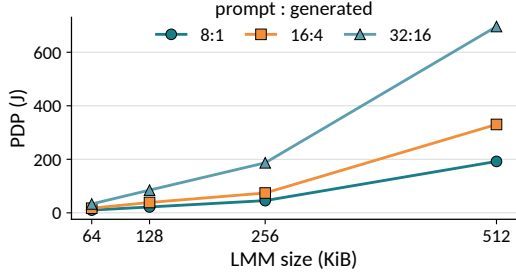
Fig. 4.7 reports PDP sensitivity across three Qwen3 models and two quantization formats. Across all 18 curves, 64KiB gives the minimum PDP. Increasing LMM capacity beyond the workload fit point raises static memory power.



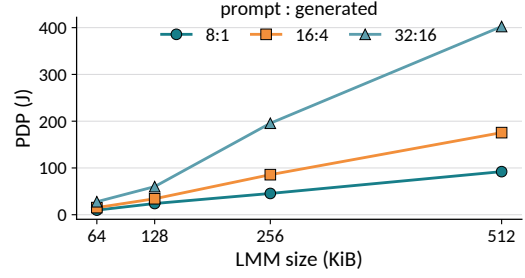
(a) Qwen3-0.6B Q3_K_S



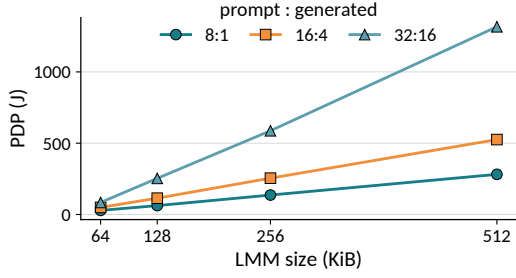
(b) Qwen3-0.6B Q8_0



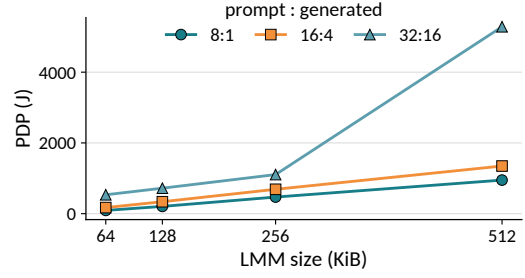
(c) Qwen3-1.7B Q3_K_S



(d) Qwen3-1.7B Q8_0



(e) Qwen3-8B Q3_K_S



(f) Qwen3-8B Q8_0

Figure 4.7. Relationship between LMM size and PDP for three Qwen3 model scales under Q3_K_S and Q8_0. Each curve corresponds to one prompt-token and generated-token pair.

5 Software Stack Improvements for CGLA

5.1 Need for Stack-Level Improvement

The measurements in Chapter 4 show that functional kernel offload does not by itself reduce end-to-end latency. The default execution policies in the OSS application produce chunk and transfer behavior mismatched with CGLA execution. Static chunking heuristics can produce chunks that are too small for burst-oriented CGLA execution. Such chunks increase kernel launches, DMA latency, and PE underutilization. Conversely, overly large chunks can exceed LMM capacity or reduce kernel coverage. Automatic mapping of arbitrary computation graphs is outside the implemented scope. On the IMAX prototype, each chunk triggers kernel configuration, LOAD, EXEC, and DRAIN. Chunk size and transfer aggregation are therefore software-stack parameters.

5.2 Q-Snap

Q-Snap is a quantization-aware dynamic chunking policy for LLM execution on a CGLA. The prompt and generated tokens reported in the evaluation are model sequence units, whereas an execution chunk is a group of matrix rows processed by a CGLA invocation. The evaluation derives chunk size from the quantization format and LMM capacity. It compares Q-Snap with the baseline and fixed chunk sizes using kernel-launch count and throughput. Q-Snap determines chunk size by considering the memory footprint of each row, the alignment constraints imposed by the quantization format, and the physical capacity of the local memory. Fig. 5.1 compares the conventional baseline, fixed-size chunking, and Q-Snap.

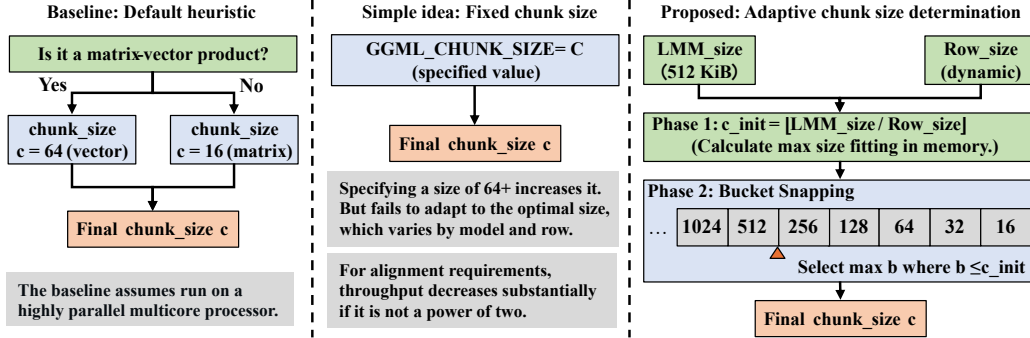


Figure 5.1. Comparison of Q-Snap, the llama.cpp default heuristic, and fixed-size chunking for the CGLA dot-product kernel.

The baseline in Fig. 5.1 uses the default llama.cpp heuristic. Q-Snap instead uses quantized row footprint and local-memory capacity, which determine burst size, DMA count, and local-memory fit.

5.2.1 Chunk-Size Derivation

Q-Snap computes chunk size as follows. Let C_{LMM} denote the available LMM capacity in bytes, and let K denote the inner-product dimension. Let b_q denote the number of elements in one block of quantization format q , and let s_q denote its size in bytes. The quantized row footprint is

$$f(q, K) = \left\lceil \frac{K}{b_q} \right\rceil s_q. \quad (5.1)$$

The maximum number of rows that fit within the LMM is given by

$$n_{\max} = \left\lfloor \frac{C_{\text{LMM}}}{f(q, K)} \right\rfloor. \quad (5.2)$$

The chunk size is then snapped down to the largest admissible power-of-two bucket so that it matches the burst-oriented transfer granularity of CGLA execution. Let $B = \{16, 32, 64, 128, 256, 512, 1024\}$ denote the bucket set. The final chunk size is

$$n_{\text{chunk}} = \max\{B_i \in B \mid B_i \leq n_{\max}\}. \quad (5.3)$$

Equation 5.3 applies when $n_{\max} \geq \min B = 16$. If $n_{\max} < 16$, no admissible IMAX bucket exists and the operation remains on the host fallback path.

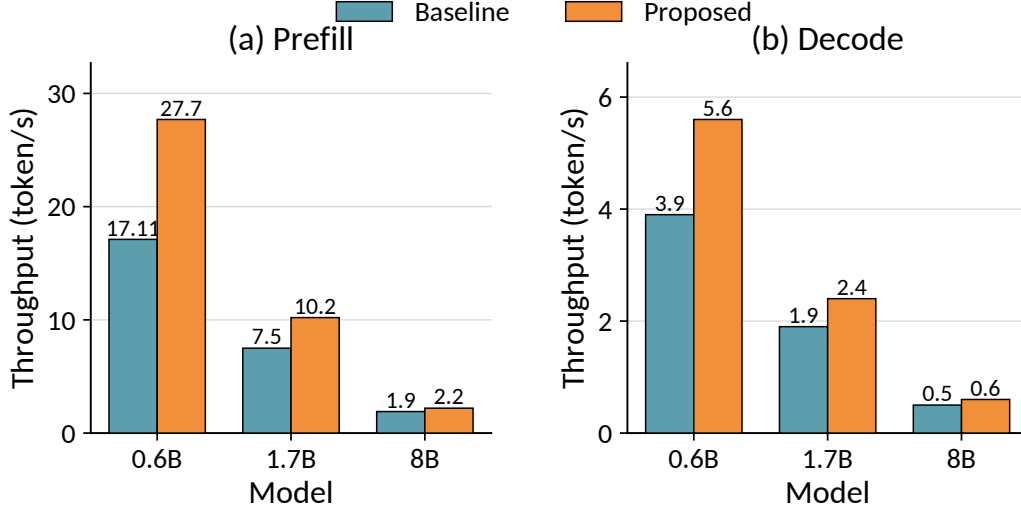


Figure 5.2. Prompt-processing and decode throughput under the baseline chunking policy and Q-Snap. Adapting chunk size to the quantization format and LMM capacity reduces kernel launches and transfer issuance.

Eq. (5.1), Eq. (5.2), and Eq. (5.3) derive chunk size from LMM capacity and the block layout of the active quantization format. The derived size changes with quantization format and LMM capacity.

The storage costs described in Chapter 4 determine $f(q, K)$ in Q-Snap. Q8_0, Q3_K, and Q6_K require 8.50, 3.44, and 6.56 bits per weight, respectively. The number of rows that fit in the same LMM therefore differs by format. Their block sizes and scale layouts determine b_q and s_q in Eq. (5.1). Q-Snap derives the chunk size from the actual row size including scale metadata rather than from the nominal weight width. The quantitative Q-Snap evaluation below uses Q3_K_S only.

5.2.2 Quantitative Results

The Q-Snap and fixed-chunk comparisons use Q3_K_S models from the Qwen3-0.6B, Qwen3-1.7B, and Qwen3-8B families. Each run uses 32 input tokens, 32 generated tokens, two host threads, and the 512 KiB LMM configuration of IMAX3. The baseline uses the default static chunking heuristic of llama.cpp. Within each comparison, only the chunk-formation policy is changed. Fig. 5.2 reports that

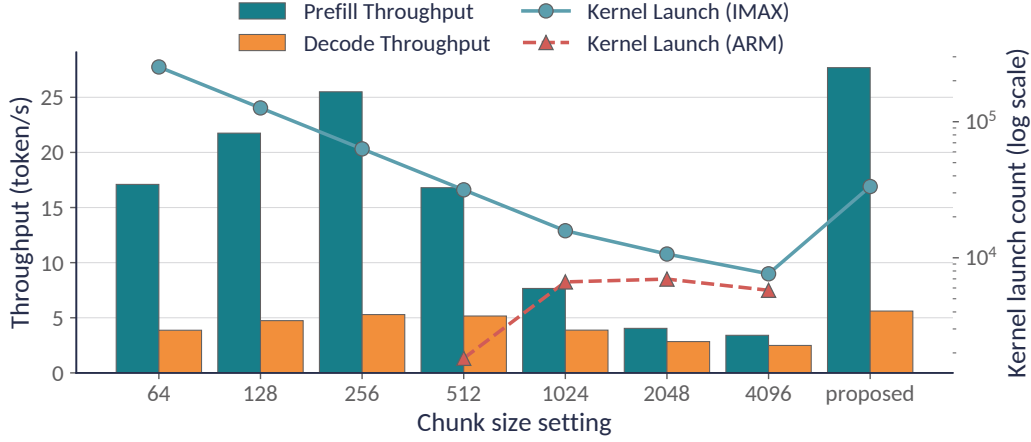


Figure 5.3. Throughput and kernel-launch count as the fixed chunk size is varied. The sweep shows the trade-off between launch overhead for small chunks and LMM incompatibility for chunks that exceed the LMM constraint.

prompt-processing throughput for Qwen3-0.6B increases from 17.11 to 27.70 tokens/s ($1.62\times$). For Qwen3-1.7B and Qwen3-8B, it increases from 7.5 to 10.2 tokens/s ($1.36\times$) and from 1.9 to 2.2 tokens/s ($1.16\times$), respectively. Decode throughput increases from 3.9 to 5.6, 1.9 to 2.4, and 0.5 to 0.6 tokens/s for the three models, respectively. The relative gain decreases for larger models as costs outside chunk formation become larger.

Figure 5.3 shows the fixed-size sensitivity sweep. Increasing the chunk size from 64 to 256 raises prompt-processing throughput from 17.10 to 25.50 tokens/s. The same change reduces IMAX kernel launches from 252,602 to 63,167. Throughput decreases to 16.80 tokens/s at 512 and to 3.41 tokens/s at 4096. Q-Snap achieves 27.69 tokens/s with 33,409 IMAX launches while preserving LMM compatibility.

The arithmetic kernel is unchanged in these comparisons. Kernel-launch counts and phase measurements therefore isolate the effect of chunk formation and transfer granularity. The next chapter applies the same backend interface to ASR and image-generation paths.

6 Speech Recognition and Image Generation on the IMAX Prototype

The CGLA backend is also integrated with Whisper ASR and Stable Diffusion on the IMAX prototype. Whisper is dominated by FP16 operations, whereas Stable Diffusion uses Q3_K, Q8_0, and FP16 dot-product offload. These data types and their kernel coverage produce different transfer and fallback behavior.

6.1 Extension to GGML-Based Applications

llama.cpp, whisper.cpp, and stable-diffusion.cpp share common GGML tensor representations, matrix-multiplication call signatures, and quantized data formats [3–5]. At the shared `mul_mat` interface, the CGLA backend reuses Q8_0, Q3_K, and Q6_K kernels where GGML formats match. It adds an FP16 kernel for Whisper and Stable Diffusion. Application-specific encoding, decoding, and denoising control remain on the host while dominant matrix operations are incrementally offloaded through this boundary.

6.2 Whisper ASR

The Whisper path uses the shared `mul_mat` interface with data-type-specific dispatch and host execution for unsupported operations. The implementation uses an FP16 dot-product kernel with inline FP16-to-FP32 conversion, SIMD operations on the 64-bit datapath, mixed execution, and padding elimination. On the host CPU, dot-product operations account for 90.6% of FP16 time and

87.1% of Q8_0 time, and these kernels are offloaded to IMAX. Whisper-tiny.en provides the main result, with base and small models used to measure the effect of model size and kernel coverage.

6.2.1 FP16 Kernel Design and Implementation Techniques

The dominant Whisper tensor format is FP16, while the LLM kernels use low-bit quantized types. The existing Q8_0 and Q3_K kernels cannot handle FP16 operations. An FP16 dot-product kernel was designed for this workload.

The FP16 kernel employs several implementation techniques. First, arithmetic logic unit 2 (ALU2) converts FP16 operands from the LMM to FP32 using bit operations. This conversion requires neither a dedicated conversion circuit nor host-side preconversion. Second, arithmetic logic unit 1 (ALU1) and the 64-bit datapath execute two 32-bit fused multiply-add (FMA) operations in parallel. Third, column-wise multithreading maps four logical FMA operations to one physical floating-point unit (FPU) and hides FPU latency in the pipeline. Operands stream from the LMM while conversion, multiplication, accumulation, and reduction proceed along the PE column, as illustrated in Fig. 6.1. The FP16 kernel employs 22 PEs per lane and maps the conversion and accumulation path onto the CGLA one-dimensional dataflow.

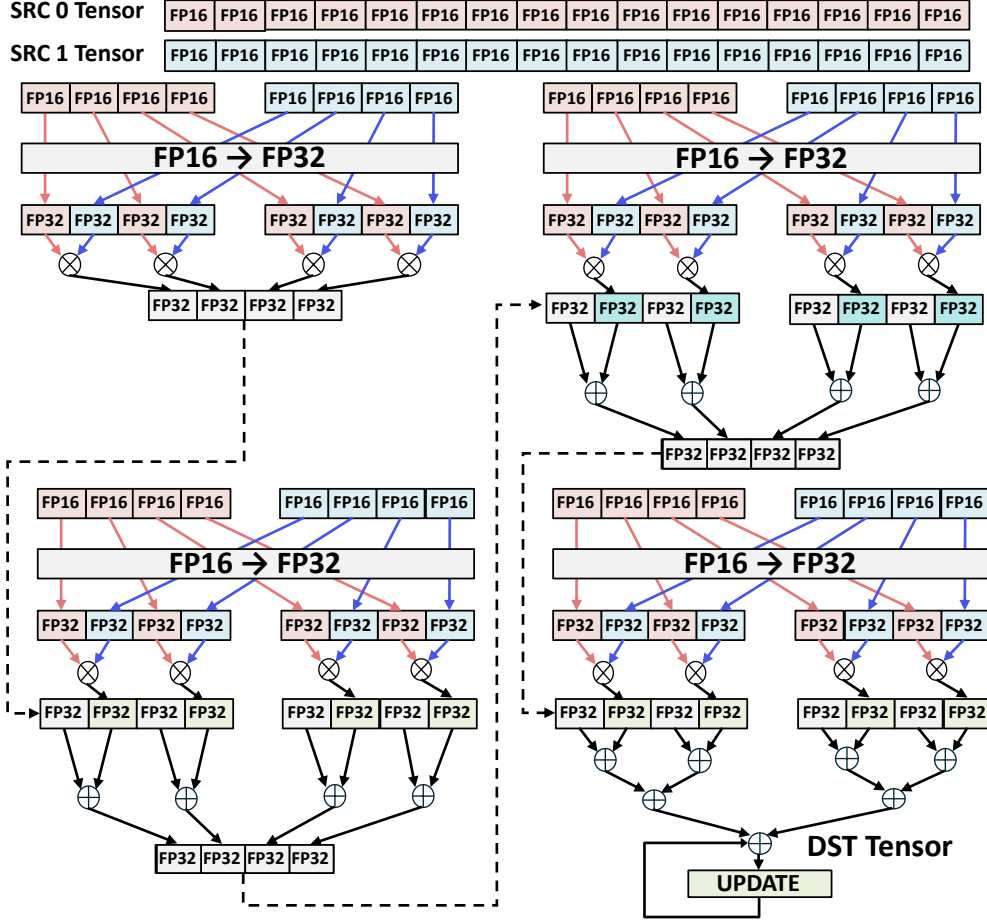


Figure 6.1. Dataflow of the FP16 dot-product kernel on the IMAX prototype. Each pair of FP16 operands from the source tensors is converted to FP32. The operands are multiplied and accumulated along the PE column. The running sum updates the destination tensor.

Fourth, a mixed execution strategy handles tensor dimensions that do not align with CGLA burst boundaries. Burst-aligned regions execute on the IMAX prototype. Residual elements execute on the host CPU. For Whisper-tiny.en, the principal matrix dimensions are exact multiples of the burst length, so the CPU-side residual is zero for these kernels. Fifth, data-padding removal reduces the LMM capacity occupied by CPU-side alignment padding. The original FP16 tensors in whisper.cpp include 32-byte alignment padding for CPU-side SIMD processing. This padding wastes LMM capacity in the CGLA data path, where

alignment constraints differ. Removing it reduces the number of padding bytes transferred to the LMM. Fig. 6.2 illustrates the Whisper ASR processing flow.

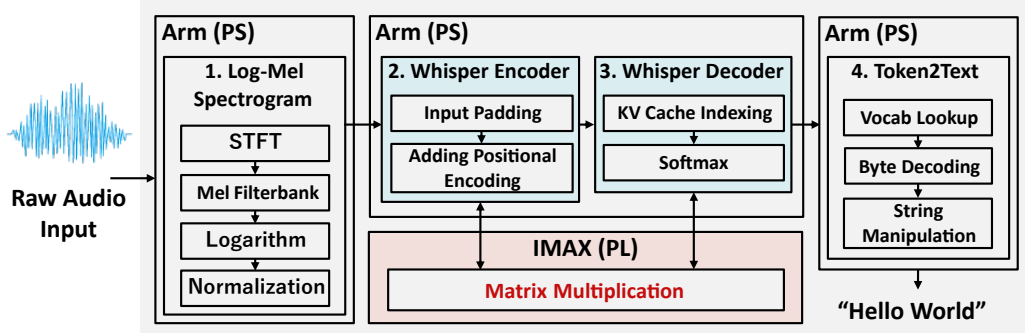


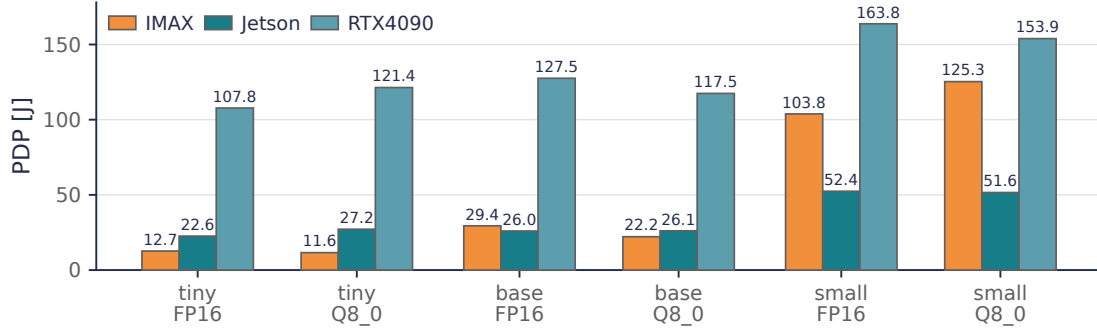
Figure 6.2. Whisper ASR processing flow. Raw audio is converted to an 80-channel log-mel spectrogram. The convolutional front end and Transformer encoder produce context embeddings. The decoder generates output tokens by attending to the encoder output and previously generated tokens.

The encoder and decoder offload FP16 dot products through the shared backend. Host-side ASR control remains unchanged. This FP16 path is also needed by Stable Diffusion, where FP16 operations dominate 59.0% of dot-product execution time.

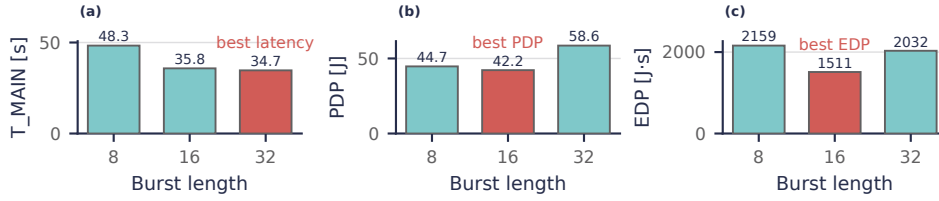
6.2.2 Energy Efficiency and Platform Comparison

The PDP values use the FPGA-to-ASIC projection and platform power estimates defined in Chapter 3. The main ASR evaluation covers three model sizes, tiny, base, and small, under the FP16 and Q8_0 formats. On 21 LibriSpeech test-clean utterances [31] with Whisper-tiny.en, the IMAX FP16 path has a mean normalized CPU-IMAX word-level transcript delta of 0.00%. Both the CPU and IMAX FP16 paths record WER 2.96% and CER 0.74%. The IMAX Q8_0 path has a mean normalized CPU-IMAX word-level transcript delta of 0.13%, and both paths record WER 5.03% and CER 1.82%.

Fig. 6.3 presents the cross-platform PDP comparison and burst-length trade-off for Whisper dot-product kernel offloading. Relative to Jetson AGX Orin, the projected IMAX has lower PDP for both tiny configurations and for base Q8_0, but higher PDP for base FP16 and both small configurations. Relative to



(a) Cross-platform PDP across Whisper model sizes.



(b) Burst-length trade-off for Whisper-tiny.en FP16.

Figure 6.3. Cross-platform PDP comparison and burst-length trade-off for Whisper ASR. The burst-length sweep uses Whisper-tiny.en FP16 with 32 KiB LMM, two lanes, and two host threads.

RTX 4090, the projected IMAX has lower PDP in all six Whisper configurations shown. In the IMAX phase measurement, EXEC accounts for 60.89% of total runtime in FP16 and 74.70% in Q8_0. In the burst-length sweep, burst length 32 minimizes latency, whereas burst length 16 gives the lowest PDP and EDP. The Whisper evaluation covers LMM capacities from 16 to 256 KiB, with the minimum PDP at 32 KiB for this workload. Smaller LMMs reduce kernel coverage and increase transfer round-trips, whereas larger LMMs increase static power beyond their coverage benefits.

Table 6.1 reports cumulative kernel coverage as a function of LMM size across the three Whisper models. At 32 KiB, coverage is 93.8% for tiny but remains near 66% for base and small, which exceed 94% only at 64 KiB.

Table 6.1. Cumulative percentage of Whisper kernels that fit within each LMM size for the tiny, base, and small models.

Model	16 KiB	32 KiB	64 KiB	128 KiB	256 KiB
tiny (78 MB)	66.4	93.8	93.8	100.0	100.0
base (148 MB)	66.5	66.5	94.2	97.1	99.9
small (488 MB)	66.5	66.5	94.4	96.9	99.9

Values are cumulative coverage percentages by kernel count.

Fig. 6.4 relates LMM capacity to projected end-to-end latency. As the LMM grows, more dot-product kernels remain on IMAX and fewer operations fall back to the CPU. The base and small models therefore benefit more from additional LMM capacity than the tiny model, whose kernel coverage is already 93.8% at 32 KiB. Kernel count alone, however, does not determine latency. For small Q8_0, coverage exceeds 94% at 64 KiB, but projected latency remains near 117s across 32–64 KiB because the remaining fallback kernels are expensive. At 128 KiB, reduced fallback time lowers latency to 46.84s.

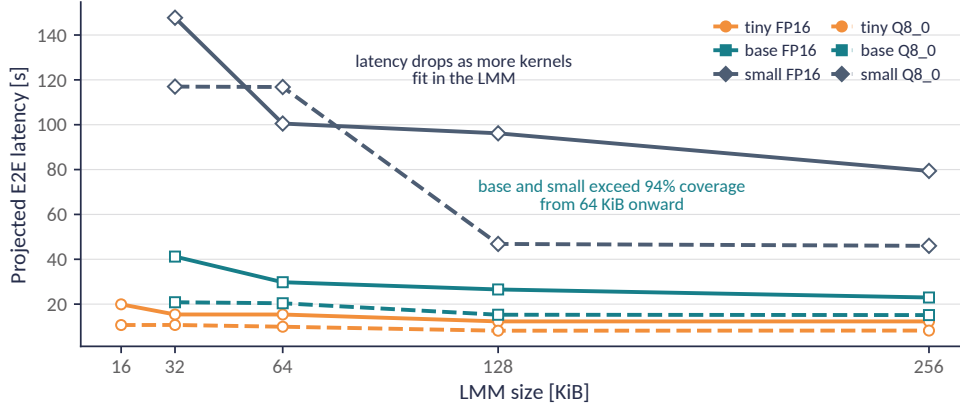


Figure 6.4. Projected end-to-end latency as a function of LMM capacity for Whisper ASR. Larger LMM capacity increases kernel coverage and reduces latency for the base and small models.

The 90.6% FP16 dot-product share gives a CPU-profile Amdahl upper bound of $10.6\times$ for ideal dot-product acceleration. The corresponding 87.1% Q8_0 share gives an upper bound of $7.8\times$. For Whisper-tiny.en FP16 with 32 KiB LMM, burst length 16 minimizes PDP and EDP, whereas burst length 32 minimizes latency at higher power. Changing the dominant data type from low-bit quantization to FP16 therefore requires an FP16 backend path and residual handling.

6.3 Stable Diffusion

The CGLA backend is integrated into `stable-diffusion.cpp` [5]. It dispatches Q3_K, Q8_0, and FP16 dot-product kernels from `mul_mat` calls while retaining the denoising pipeline on the host.

6.3.1 Experimental Setup and Quantitative Results

The Stable Diffusion evaluation employed IMAX3 in a single-lane configuration at 145 MHz with 512 KiB LMM per lane. The target model is SD-Turbo, which is based on Adversarial Diffusion Distillation for one- to four-step sampling [32]. The platform comparison uses Q3_K and Q8_0. The execution path also dispatches FP16 dot-product operations through the same CGLA backend. The IMAX ASIC values use the projection defined in Chapter 3.

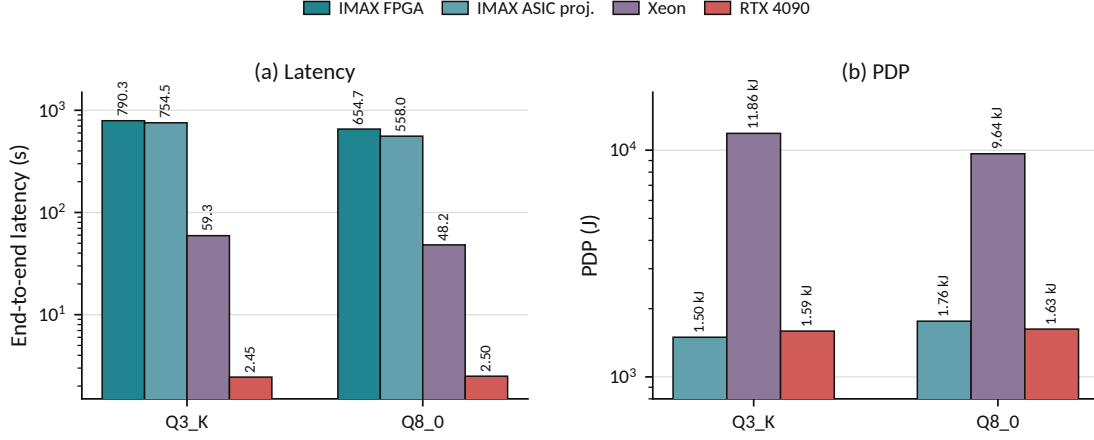


Figure 6.5. End-to-end latency and PDP for SD-Turbo on IMAX FPGA, the projected 28 nm ASIC, Xeon, and RTX 4090.

The RTX 4090 evaluation uses SD-Turbo at 512×512 resolution with one denoising step. Five runs were performed for each of Q3_K and Q8_0. The median elapsed times were 2.45s for Q3_K and 2.50s for Q8_0. Figure 6.5 compares IMAX FPGA, the projected ASIC, Xeon, and RTX 4090 measurements. The RTX 4090 PDP uses a nominal platform power estimate [27, 28].

RTX 4090 has the shortest end-to-end latency for both quantization formats in Fig. 6.5. The projected IMAX ASIC has lower PDP than RTX 4090 for Q3_K but higher PDP for Q8_0. The ordering changes because the quantization formats alter the offloaded kernel mix, transfer volume, and host execution that remain in the complete image-generation path.

The workload composition explains why Stable Diffusion uses the FP16 kernel alongside quantized kernels for image generation. Q3_K, FP16, and FP32 represent 10.3%, 59.0%, and 30.7% of Stable Diffusion dot-product execution time, respectively. The Whisper FP16 kernel was compiled against the Stable Diffusion tensor layout. It was compared with the CPU reference at $K = 256, 320, 640$, and 1280 with output-column counts of 1 and 16. These tests matched the CPU reference without guard-region corruption.

6.4 Cross-Workload Discussion

LLM inference, Whisper ASR, and Stable Diffusion share the GGML `mul_mat` interface but have different performance constraints. The constraints are quantization and repeated decode transfers for LLMs, FP16 and residual handling for Whisper, and FP16/FP32 coverage for Stable Diffusion. The shared interface limits code changes to backend dispatch and data transfer. These policies still account for each workload’s dominant data type, tensor shape, and burst constraints. For these workloads, CGLA offload applies to dominant kernels that match linear dataflow. Their input and intermediate data must be partitionable into blocks that fit LMM capacity. LMM capacity is compared using kernel coverage and memory power for each workload.

6.5 Evaluation Scope and Limitations

PDP and EDP use synthesis-based ASIC estimates and nominal platform power estimates instead of workload-average power measurements. The FPGA-to-ASIC projection scales IMAX-active time to the 840 MHz Design Compiler condition. Host CPU time remains unscaled.

The Whisper accuracy check uses 21 LibriSpeech test-clean utterances. The latency and PDP comparison uses one standardized speech clip of approximately ten seconds. This input controls the platform comparison but does not measure input-length variability.

The workload set covers GGML-based applications centered on matrix operations. Query-dependent irregular memory access and search data structures are not measured.

7 Conclusion

This thesis implemented and evaluated CGLA execution paths for LLM inference, Whisper ASR, and Stable Diffusion on IMAX3. Across these low-batch application paths, input transfer, host processing, uncovered kernels, and residual FP32 or application control constrained performance. PE arithmetic was not the dominant cost in these cases. The three applications retain their original model loading, tensor management, command-line interfaces, and application control while the GGML backend selects CGLA or CPU execution.

In the llama.cpp path, data transfer and host-side overhead limit decode execution. Q-Snap increased prompt-processing throughput by a factor of 1.62. For Whisper-tiny.en, the FP16 and Q8_0 paths record the same WER and CER as their CPU counterparts on the 21-utterance accuracy set. Relative to Jetson AGX Orin, the projected IMAX has lower PDP for both tiny configurations and for base Q8_0, but higher PDP for base FP16 and both small configurations. Relative to RTX 4090, the projected IMAX has lower PDP in all six Whisper configurations shown. For SD-Turbo, the projected IMAX ASIC PDP ordering relative to RTX 4090 depends on quantization format, while the tested FP16 shapes match the CPU reference without guard-region corruption.

Future work includes a higher-bandwidth host interconnect for the host- and transfer-bound decode phase and native support for additional low-bit quantization formats. Key-value cache handling, kernel-mapping tools, and evaluation with additional models and inputs also remain.

Acknowledgements

I thank Professor Yasuhiko Nakashima for his guidance and support throughout my master's research. I also thank Professor Yuichi Hayashi for reviewing this thesis and providing comments on the research. I thank Visiting Professor Renyuan Zhang, Assistant Professor Yirong Kan, Assistant Professor Pham Hoai Luan, and Assistant Professor Le Vu Trung Duong for their advice and encouragement.

I thank the members of the Nakashima Laboratory for the research environment and discussions that supported this work. I thank Assistant Professor Reon Oshio (Tokyo Metropolitan University), formerly a member of the Nakashima Laboratory, for the insightful discussions and lectures that informed this research. I thank Yu Eto, Ayumu Takeuchi, and Yoshifumi Munakata for their collaboration on IMAX, CGLA systems, and AI workloads. I also thank the senior and junior students in the laboratory for their support during my graduate studies.

I thank my partner, Hina Sengoku, for her support throughout this period. Finally, I thank my family for their understanding and support.

References

- [1] T. Akabe, V. T. D. Le, and Y. Nakashima, “IMAX: A power-efficient multilevel pipelined CGLA and applications,” *IEEE Access*, vol. 13, pp. 31899–31911, 2025.
- [2] A. Podobas, K. Sano, and S. Matsuoka, “A survey on coarse-grained reconfigurable architectures from a performance perspective,” *IEEE Access*, vol. 8, pp. 146719–146743, 2020.
- [3] ggml-org, “llama.cpp: LLM inference in C/C++.” <https://github.com/ggml-org/llama.cpp>, 2026. Accessed: July 23, 2026.
- [4] ggml-org, “whisper.cpp: Port of OpenAI’s Whisper model in C/C++.” <https://github.com/ggml-org/whisper.cpp>, 2026. Accessed: July 23, 2026.
- [5] leejet, “stable-diffusion.cpp: Diffusion model inference in C/C++.” <https://github.com/leejet/stable-diffusion.cpp>, 2026. Accessed: July 23, 2026.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30*, pp. 5998–6008, 2017.
- [7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems 33*, 2020.

- [8] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 30318–30332, 2022.
- [10] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [11] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” in *Low-Power Computer Vision*, pp. 291–326, Chapman and Hall/CRC, 2022.
- [12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, pp. 28492–28518, PMLR, 2023.
- [13] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10674–10685, 2022.
- [14] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [15] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, “EIE: Efficient inference engine on compressed deep neural network,” in *Proceedings of the 43rd Annual International Symposium on Computer Architecture*, pp. 243–254, 2016.

- [16] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [17] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, “Plasticine: A reconfigurable architecture for parallel patterns,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pp. 389–402, 2017.
- [18] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-L. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pp. 1–12, 2017.
- [19] M. Tanomoto, S. Takamaeda-Yamazaki, J. Yao, and Y. Nakashima, “A CGRA-based approach for accelerating convolutional neural networks,” in *2015 IEEE 9th International Symposium on Embedded Multicore/Many-core Systems-on-Chip*, pp. 73–80, 2015.
- [20] J. Iwamoto, Y. Kikutani, R. Zhang, and Y. Nakashima, “Daisy-chained systolic array and reconfigurable memory space for narrow memory bandwidth,” *IEICE Transactions on Information and Systems*, vol. E103-D, no. 3, pp. 578–589, 2020.

- [21] Advanced Micro Devices, Inc., “VPK180 evaluation board user guide: Versal device.” <https://docs.amd.com/r/en-US/ug1582-vpk180-eval-bd/Versal-Device>, 2025. Accessed: July 23, 2026.
- [22] Synopsys, Inc., “DC Ultra: Concurrent timing, area, power and test optimization.” <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html>. Accessed: May 25, 2025.
- [23] H. Uetani and Y. Nakashima, “Implementation and evaluation of LLM on a CGLA,” in *2024 Twelfth International Symposium on Computing and Networking (CANDAR)*, pp. 252–258, 2024.
- [24] Y. Eto and Y. Nakashima, “Implementation and performance analysis of LLaMA on a CGLA,” in *Proceedings of the Fifth International Conference on Intelligent Systems and Networks*, pp. 516–525, Springer Nature Singapore, 2025.
- [25] V. J. Reddi *et al.*, “MLPerf inference benchmark,” in *Proceedings of the 47th Annual International Symposium on Computer Architecture*, pp. 446–459, 2020.
- [26] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [27] NVIDIA Corporation, “GeForce RTX 4090 specifications.” <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/>, 2022. Accessed: July 23, 2026.
- [28] Intel Corporation, “Intel Xeon w5-2455X Processor specifications.” <https://www.intel.com/content/www/us/en/products/sku/233420/intel-xeon-w52455x-processor-30m-cache-3-20-ghz/specifications.html>, 2023. Accessed: July 23, 2026.
- [29] NVIDIA Corporation, “Introducing the GeForce GTX 1080 Ti, the world’s fastest gaming GPU.” <https://www.nvidia.com/en-us/geforce/news/nvidia-geforce-gtx-1080-ti/>, 2017. Accessed: August 20, 2026.

- [30] NVIDIA Corporation, “NVIDIA Jetson AGX Orin Series technical brief,” tech. rep., NVIDIA Corporation, 2022. Accessed: July 23, 2026.
- [31] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5206–5210, 2015.
- [32] A. Sauer, D. Lorenz, A. Blattmann, and R. Rombach, “Adversarial diffusion distillation,” *arXiv preprint arXiv:2311.17042*, 2023.

A Publications

Peer-reviewed journal papers

1. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima, “Efficient Kernel Mapping and Comprehensive System Evaluation of LLM Acceleration on a CGLA,” *IEEE Access*, vol. 13, pp. 199631–199646, November 2025.
2. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima, “Design and Evaluation of Energy-Efficient Whisper Dot-Product Kernel Offloading on a CGLA Architecture,” *Concurrency and Computation: Practice and Experience*, vol. 38, no. 13, article e70826, 2026.

Peer-reviewed conference papers

1. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima, “Energy-Efficient Llama3 Acceleration on a CGLA by Offloading Computational Kernels,” *Proceedings of the 2025 IEEE 38th International System-on-Chip Conference (SOCC) PhD and Master Forum*, September 2025.
2. Yu Eto, Takuto Ando, Ayumu Takeuchi, and Yasuhiko Nakashima, “LLM Performance Bottlenecks on CGLA,” *Proceedings of the 2025 IEEE 38th International System-on-Chip Conference (SOCC) PhD and Master Forum*, September 2025.
3. Ayumu Takeuchi, Yu Eto, Takuto Ando, and Yasuhiko Nakashima, “Energy-Efficient FlashAttention Acceleration on CGLA,” *Proceedings of the 2025 IEEE 38th International System-on-Chip Conference (SOCC) PhD and Master Forum*, September 2025.
4. Takuto Ando, Yu Eto, and Yasuhiko Nakashima, “A Detailed Analysis of LLM Execution on IMAX3 and Initial Evaluation of IMAX4 Prototype for Server Environment,” *Proc. SASIMI 2025*, pp. 8–13, October 2025.
5. Yu Eto, Takuto Ando, and Yasuhiko Nakashima, “Performance Evaluation of Flan-T5 in CGLA Based Accelerators,” *Proceedings of the 2025 IEEE 38th International System-on-Chip Conference (SOCC)*, October 2025.
6. Takuto Ando, Yu Eto, Ayumu Takeuchi, and Yasuhiko Nakashima,

“Energy-Efficient Hardware Acceleration of Whisper ASR on a CGLA,”
*Proceedings of the 2025 Thirteenth International Symposium on Computing and
Networking (CANDAR 2025)*, November 2025.

7. Takuto Ando, Yu Eto, and Yasuhiko Nakashima, “Implementation and
Evaluation of Stable Diffusion on a General-Purpose CGLA Accelerator,”
*Proceedings of the 2025 IEEE 18th International Symposium on Embedded
Multicore/Many-core Systems-on-Chip (MCSoc)*, December 2025.
8. Takuto Ando, Ayumu Takeuchi, Yu Eto, Yoshifumi Munakata, and Yasuhiko
Nakashima, “Q-Snap: Quantization-Aware Dynamic Chunking for LLM
Execution on a CGLA,” *Proceedings of the Sixth International Conference on
Intelligent Systems and Networks (ICISN 2026)*, March 2026.

Domestic conference papers (in Japanese)

1. Takuto Ando, Yoshifumi Munakata, and Yasuhiko Nakashima,
“Implementation and Future Prospects of an Integrated Software Stack for
Diverse AI Workloads on CGLA (in Japanese),” *Proceedings of the 257th
System and Architecture Research Symposium*, June 2026.
2. Yoshifumi Munakata, Takuto Ando, and Yasuhiko Nakashima, “Scalability
Evaluation of CGLA for LLMs Using High-Bandwidth Memory (in Japanese),”
Proceedings of the 257th System and Architecture Research Symposium, June
2026.

B Awards

1. IEEE CEDA All Japan Joint Chapter SASIMI Young Researcher Award, SASIMI
2025, October 2025.
2. Best Paper Award, CANDAR 2025, November 2025.
3. Best Paper Award, ICISN 2026, March 2026.